ICSLab2

1. Q1

1.1 破题思路

第一眼看到如此复杂的汇编代码，冲击力很大，与课上较为简单的汇编形成了鲜明对比，所以我决定重点关注这个代码的结构，从而剖析每一部分都在做什么。

题目中有很多是赋值与进栈退栈，对于剖析结构没有很大作用，我重点寻找函数调用、条件跳转和返回值判断，确定程序的整体流程。在 `phase_01` 中，利用`gdb`可以发现三个重要的函数调用：`valid_u32_fields`：检查输入格式；`sscanf`：从输入字符串中提取整数；`p1_compute`：根据 `challenge` 计算用于验证的数值。进一步观察 `cmp` 和 `sete` 指令，可以判断最终的验证条件是输入整数等于 `p1_compute` 的返回值。因此，破题的关键转化为：通过反汇编还原 `p1_compute` 的计算公式，并在求解器中实现相同的运算。

1.2 结构剖析

接下来就是结构剖析，根据 `phase_01` 的汇编，可以将整个验证过程划分为四个部分：

1. 保存输入参数并建立栈帧。
2. 检查输入格式是否合法。
3. 解析输入字符串，获取整数。
4. 计算期望答案，并与输入进行比较。

其中，程序首先通过 `mov` 保存输入字符串和 `challenge`，再利用 `push`、`sub` 等指令维护栈空间。这些操作主要用于寄存器保护和函数参数传递，并不直接参与答案计算。

用一个较为粗糙的流程图，过程可以体现为：

```
输入 input 和 challenge
          |
  valid_u32_fields
          |
     格式是否合法?
      /         \
     否          是
     |           |
   返回0       sscanf
               |
          解析是否成功?
            /      \
           否       是
           |        |
         返回0   p1_compute
                    |
                计算期望答案
                    |
                与输入比较
                    |
                返回结果
```

由此可见，第一关的核心计算集中在 `p1_compute` 中，其他部分主要负责输入检查和结果判断。

1.3 各部分逐步突破

1.3.1 输入格式检查

```
mov $0x1,%esi
call valid_u32_fields
test %al,%al
jne 402050
```

Fence 2

`%esi` 被设置为 1，表示要求一个合法的无符号 32 位整数字段。`test` 检查函数返回值，若非零则通过检查，继续执行。

1.3.2 输入整数解析

```
lea -0x14(%rbp),%rdx
mov $0x403182,%esi
mov %rbx,%rdi
call __isoc99_sscanf@plt
cmp $0x1,%eax
jne 402043
```

Fence 3

这里按照 x86-64 调用约定，将输入字符串、格式字符串和整数存储地址传递给 `sscanf`。

随后比较返回值与 1，确保成功读取一个整数。解析结果保存在栈上的局部变量中。

1.3.3 最终验证条件

```
mov %r12,%rdi
call p1_compute
cmp %eax,-0x14(%rbp)
sete %al
```

Fence 4

`%r12` 中保存的是 `challenge`，将其传递给 `p1_compute` 后，计算结果通过 `%eax` 返回。

`cmp` 比较计算结果与输入整数，`sete` 在两者相等时设置 `%al = 1`，否则设置为 0。

因此，可以得到第一关的核心验证条件：

```
input_value == p1_compute(challenge)
```

Fence 5

1.3.4 计算函数 p1_compute

反汇编 `p1_compute`，发现它首先通过 `mov` 和 `shr` 指令将 64 位挑战值拆分为三个 32 位数据：

```
uint32_t low = (uint32_t)challenge;
uint32_t mid = (uint32_t)(challenge >> 16);
uint32_t high = (uint32_t)(challenge >> 32);
```

Fence 6

随后依次调用 `p1_step2`、`p1_step0` 和 `p1_step1`。

通过继续反汇编三个子函数，可以分别还原其运算关系。

`p1_step2`：

```
lea (%rdi,%rdi,2),%eax
add %esi,%eax
xor $0x3b60,%eax
```

Fence 7

计算：

```
a = (3u * high + low) ^ 0x3b60u;
```

Fence 8

`p1_step0`：

```
lea 0xc50(%rsi),%eax
xor %edi,%eax
```

Fence 9

计算：

```
b = (mid + 0xc50u) ^ low;
```

Fence 10

`p1_step1`：

```
xor $0x121e,%esi
lea (%rsi,%rdi,8),%eax
```

Fence 11

将前两个计算结果组合，得到：

```
answer = (a ^ 0x121eu) + 8u * b;
```

Fence 12

至此，`p1_compute` 的完整计算规则已经确定。

1.4 最终代码实现

根据逆向得到的计算公式，在 `solve_phase_1` 中直接使用 `challenge` 计算答案：

```c
int solve_phase_1(uint64_t challenge, char *out, size_t cap)
{
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;
    uint32_t low = (uint32_t)challenge;
    uint32_t mid = (uint32_t)(challenge >> 16);
    uint32_t high = (uint32_t)(challenge >> 32);
    uint32_t a = (3u * high + low) ^ 0x3b60u;
    uint32_t b = (mid + 0xc50u) ^ low;
    values[0] = (a ^ 0x121eu) + 8u * b;
    return write_answer(out, cap, values, count);
}
```

Fence 13

由于只需要输出一个整数，因此设置 `count = 1`。最后调用 `write_answer` 将计算结果输出。

1.5 运行结果截图

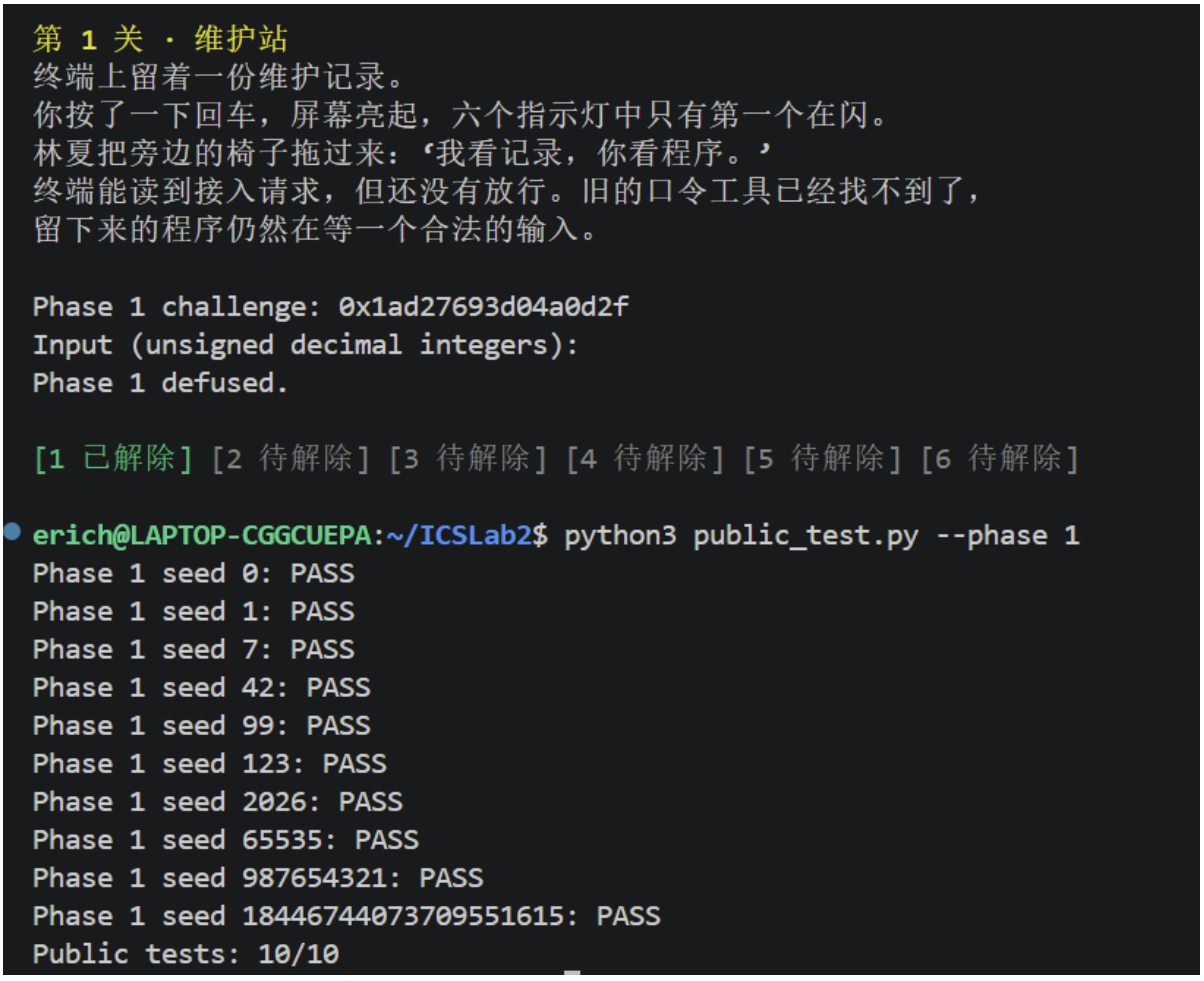


Figure 1

2. Q2

2.1 破题思路

这道题竟然比上一道题的汇编代码更长，但是很多地方与上一题一致，大部分维护栈空间的代码对于破题并没有起到关键作用。随后我惊喜的发现程序使用了多个 `lea`，对应的栈地址从 `-0x40(%rbp)` 到 `-0x2c(%rbp)`，相邻地址均相差 4 字节。考虑到 `uint32_t` 恰好占 4 字节，推测程序使用了一个包含多个整数的数组。

发现以下指令：

```
mov -0x40(%rbp,%rdx,4),%eax
```

Fence 14

其访问地址为 `rbp-0x40+rdx*4`，符合数组按下标访问的特点，进一步验证了这一猜测。

接着通过 `cmp`、`jne` 等指令识别出循环结构。分析发现，程序先验证数组首项，再循环验证其余元素。

因此，本关的关键在于：确定数组首项与 `challenge` 的关系，并还原后续元素的递推公式。

2.2 结构剖析

根据反汇编结果，可以将 `phase_02` 划分为四个部分：

1. 调用 `valid_u32_fields`，验证输入格式。
2. 使用 `sscanf` 读取整数并存入栈空间。
3. 根据 `challenge` 计算第一个整数的预期值。
4. 利用前一项、循环下标及 `challenge` 计算下一项并进行比较。

首先观察：

```
mov $0x6,%esi
call valid_u32_fields
```

Fence 15

可以判断本关要求输入 6 个无符号 32 位整数。

结合后续数组式访问，能够将其理解为：

```
uint32_t a[6];
```

Fence 16

验证过程实际上是先单独检查 `a[0]`，随后通过循环依次检查 `a[1]` 至 `a[5]`。任何一次比较失败都会跳转至失败分支，全部满足要求才会通过。

2.3 各部分逐步突破

2.3.1 输入解析与数组结构

`sscanf` 调用前存在多条 `lea` 指令，用于计算整数的存储地址：

```
lea -0x40(%rbp),%rdx
lea -0x3c(%rbp),%rcx
lea -0x38(%rbp),%r8
lea -0x34(%rbp),%r9
```

Fence 17

另外两个地址通过栈传递。最终六个整数依次存储在：

```
a[0] : rbp - 0x40
a[1] : rbp - 0x3c
a[2] : rbp - 0x38
a[3] : rbp - 0x34
a[4] : rbp - 0x30
a[5] : rbp - 0x2c
```

Fence 18

随后程序通过 `cmp $0x6,%eax` 检查 `sscanf` 是否成功解析了 6 个整数，我先记录下了这几个数及其对应地址，以便于后面的解题。

2.3.2 数组首项的确定

找到首项验证部分：

```
mov %r13,%rdx
mov -0x40(%rbp),%eax
shr $0x24,%rdx
movzbl %dl,%edx
lea (%rdx,%rdx,2),%edx
cmp %edx,%eax
jne 4020a5
```

Fence 19

其中 `%r13` 保存 `challenge`，`%eax` 保存数组首项 `a[0]`。

首先通过 `shr $0x24,%rdx` 将 `challenge` 右移 36 位，再通过 `movzbl` 提取结果的低 8 位。

接着 `lea (%rdx,%rdx,2),%edx` 计算原数的 3 倍，最后与 `a[0]` 比较。

用人话来说就是challenge右移36位然后保留最低八位再乘以3。

因此得到首项公式：

$$a_0 = 3 \times ((challenge \gg 36) \& \ 0xff)$$

如果两者不相等，则验证失败。

2.3.3 循环结构分析

继续观察后续代码，发现循环初始化和结束条件：

```
mov $0x10,%ecx
mov $0x1,%edx

...

add $0x1,%rdx
add $0x5,%ecx
cmp $0x6,%rdx
jne 402108
```

Fence 20

由此可知，`%rdx` 作为循环下标，从 1 开始，每轮增加 1，直到等于 6 时退出。

同时，`%ecx` 从 16 开始，每轮增加 5，用于控制后续的右移位数。

因此，循环共执行 5 次，依次验证 `a[1]` 到 `a[5]`。

2.3.4 递推关系的还原

循环内部最关键的指令如下：

```
mov %r13,%rdi
add %edx,%eax
shr %cl,%rdi
lea (%rax,%rax,4),%eax
and $0x3f,%edi
sub %edi,%eax
```

Fence 21

这里我特别注意 `%eax` 的数据变化。

第一次进入循环时，`%eax` 保存 `a[0]`，`%edx` 为 1。

`add %edx,%eax` 将下标加到上一项上，而 `lea (%rax,%rax,4),%eax` 实际计算的是原数的 5 倍，并不是访问内存，一开始我误以为是访问内存，后面发现想错了。

因此，这两步完成：

$$5(a_{i-1}+i)$$

另一方面，程序通过 `shr %cl,%rdi` 将 `challenge` 右移指定的位数，再通过 `and $0x3f,%edi` 保留结果的低 6 位。

由于初始移位量为 16，每轮增加 5，因此第 (i) 轮的移位量为：

$$s_i = 16 + 5(i-1)$$

最后通过 `sub %edi,%eax`，减去从 `challenge` 中提取的数值。

继续观察比较部分：

```
mov %eax,%esi
mov -0x40(%rbp,%rdx,4),%eax
cmp %esi,%eax
jne 4020a5
```

Fence 22

程序先保存计算结果，再读取当前数组元素 `a[i]`，比较两者是否相等。

同时，当前元素被读取到 `%eax` 后，会在下一轮循环中作为上一项参与计算。

因此，最终恢复出的递推关系为：

$$\boxed{a_i = 5(a_{i-1}+i) - ((challenge \gg (11+5i)) \& \ 0x3f)}$$

其中 (i=1,2,3,4,5)，运算结果按照 32 位无符号整数规则处理。

至此，本关的完整数列生成规律已经确定，接下来是轻松愉快的转c语言环节。

2.4 最终代码实现

根据上述分析，先由 `challenge` 计算首项，再使用 `for` 循环生成后续五项。

```
int solve_phase_2(uint64_t challenge, char *out, size_t cap)
{
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 6;
    values[0] = 3u * ((challenge >> 36) & 0xffu);
    for (int i = 1; i < 6; i++) {
        uint32_t shift = 16 + 5 * (i - 1);
        uint32_t temp = (challenge >> shift) & 0x3fu;
        values[i] = 5u * (values[i - 1] + (uint32_t)i) - temp;
    }
    return write_answer(out, cap, values, count);
}
```

Fence 23

本关要求输出 6 个整数，因此设置 `count = 6`，通过 `write_answer` 将数组转换为符合要求的输出。

2.5 运行结果截图

```
单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

erich@LAPTOP-CGGCUEPA:~/ICSLab2$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

Figure 2

3. Q3

3.1 破题思路

这道题的题面看起来友好了很多，同时由于已经完成前两关，我逐渐能阅读汇编语言，因此很快注意到了关键函数 `p3_dispatch`。

从 `valid_u32_fields` 和 `sscanf` 可以判断本关要求输入两个整数。进一步观察发现，第一个整数通过简单的位运算验证，而第二个整数由 `p3_dispatch` 的返回值决定。

因此，本关的思路比较明确：先还原两个输入整数与 `challenge` 的关系，再重点分析 `p3_dispatch` 的分支结构及计算方式，毕竟手册里就写了这个是switch分支有关的题目。

3.2 结构剖析

根据汇编，`phase_03` 的验证过程可以分为四部分：

1. 检查并解析两个无符号 32 位整数。
2. 从 `challenge` 中提取低 3 位，验证第一个整数。
3. 对 `challenge` 进行移位和异或，得到 `p3_dispatch` 的计算参数。
4. 根据分支编号执行相应计算，将结果与第二个整数比较。

其中前两部分相对简单，关键在于 `p3_dispatch` 如何选择不同分支。

3.3 各部分逐步突破

3.3.1 输入检查与第一个整数

```
mov $0x2,%esi
call valid_u32_fields
```

Fence 24

结合后续 `sscanf` 返回值与 2 的比较，确定需要输入两个整数，记为 `a` 和 `b`。

```
mov %r12d,%edi
xor %eax,%eax
and $0x7,%edi
cmp %edi,-0x18(%rbp)
jne 402265
```

Fence 25

其中 `%r12` 保存 `challenge`，`and $0x7,%edi` 保留其最低 3 位，再与第一个输入整数比较。

因此：$a = challenge \& 7$ 显然，`a` 的取值范围为 0 ~ 7。

3.3.2 计算第二个参数

```
mov %r12,%rax
mov %r12,%rsi
shr $0x3,%rax
shr $0x20,%rsi
xor %eax,%esi
call p3_dispatch
```

这里将 `challenge` 分别右移 3 位和 32 位，再对所得结果的低 32 位进行异或。

令：

$x = (uint32_t)(challenge \gg 3) \oplus (uint32_t)(challenge \gg 32)$

因此，`p3_dispatch` 接收的两个参数分别是 `a` 和 `x`。

3.3.3 跳转表分析

进入 `p3_dispatch` 后，我很快发现了以下指令：

```
cmp $0x7,%eax
ja 402235
notrack jmp *0x4057e0(,%rax,8)
```

Fence 27

这里先判断分支编号是否超过 7，然后根据 `%rax` 的值进行间接跳转。

跳转目标的存储地址为：

$0x4057e0 + 8a$

由此判断该函数使用了跳转表，对应 C 语言中的 `switch-case` 结构。

为了确定每个编号对应的实际分支，在 GDB 中执行：

```
x/8ga 0x4057e0
```

Fence 28

得到以下对应关系：

分支编号 a	实际执行函数
0	`p3_case_7`
1	`p3_case_4`
2	`p3_case_1`
3	`p3_case_6`
4	`p3_case_0`
5	`p3_case_2`
6	`p3_case_3`
7	`p3_case_5`

Table 1

这说明跳转表的索引顺序与 `p3_case` 的编号顺序并不一致，需要按照表中的实际地址确定执行的函数。

3.3.4 各分支计算关系

由于八个 `p3_case` 函数的汇编都较短，符合课上例题难度，主要由 `lea`、`xor`、`shr` 和 `add` 等指令组成，因此逐个反汇编后，很容易将其还原为 C 语言计算表达式。

例如 `p3_case_0`：

```
lea 0x0(,%rdi,4),%eax
xor $0xb24e,%eax
ret
```

Fence 29

先计算参数的 4 倍，再与 `0xb24e` 异或：

```
return (4u * x) ^ 0xb24eu;
```

Fence 30

其余函数按照相同方法分析，结合跳转表后，整理出完整计算关系：

a	第二个整数 b
0	`3*x + 0x671b`
1	`5*x + 0x1946a`
2	`5*x + 0x4b479`
3	`(x ^ 0xc26e) + (x >> 1)`
4	`(4*x) ^ 0xb24e`
5	`(x ^ 0x3862) + (x >> 2)`
6	`3*x + 0x2d4e`
7	`(2*x) ^ 0xf035`

Table 2

至此，第三关的验证逻辑已经完整还原：第一个整数决定分支编号，第二个整数由对应分支的计算结果确定。

3.4 最终代码实现

根据上述关系，使用 `switch-case` 直接生成两个整数：

```
int solve_phase_3(uint64_t challenge, char *out, size_t cap)
{
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;
    uint32_t a = (uint32_t)challenge & 7u;
    uint32_t x = (uint32_t)(challenge >> 3)
               ^ (uint32_t)(challenge >> 32);
    values[0] = a;
    switch (a) {
        case 0: values[1] = 3u * x + 0x671bu; break;
```

```
            case 1: values[1] = 5u * x + 0x1946au; break;
            case 2: values[1] = 5u * x + 0x4b479u; break;
            case 3: values[1] = (x ^ 0xc26eu) + (x >> 1); break;
            case 4: values[1] = (4u * x) ^ 0xb24eu; break;
            case 5: values[1] = (x ^ 0x3862u) + (x >> 2); break;
            case 6: values[1] = 3u * x + 0x2d4eu; break;
            case 7: values[1] = (2u * x) ^ 0xf035u; break;
        }
        return write_answer(out, cap, values, count);
    }
```

Fence 31

3.5 运行结果截图

```
第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0xf5496a1496a94cc6
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]

erich@LAPTOP-CGGCUEPA:~/ICSLab2$ python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

Figure 3

4. Q4

4.1 破题思路

做完三题的我已经有了经验，根据 `valid_u32_fields` 的参数可以判断，本关只需要输入一个整数，而主要计算集中在 `p4_walk` 函数中。

于是继续反汇编 `p4_walk`。一开始尝试还原函数前面的几条指令，发现程序先计算两个参数的差值，右移一位后再加上其中一个参数。在草稿上写成人类的语言，发现这正是二分查找中计算区间中点的公式。

因此我很快判断或者说猜测出 `p4_walk` 应该是一个二分查找函数，同时我还猜测会有 > target和 < target的不同分支。

我欣喜的观察到了后面的 `cmp`、`je` 和 `jb` 指令，发现程序会根据目标值与中点的大小关系进入不同分支。结合两处对 `p4_walk` 自身的调用，可以进一步确定这是一个递归二分查找过程。

确定基本算法后，剩下只需要分别还原相等、大于和小于中点时的处理逻辑，以及递归返回后对结果的运算。

4.2 程序结构剖析

整个 `phase_04` 可以分成三个部分：

1. 通过 `valid_u32_fields` 和 `sscanf` 读取一个无符号 32 位整数。
2. 对 `challenge` 进行移位、按位与和异或，构造 `p4_walk` 的参数。
3. 调用 `p4_walk`，并将返回值与输入整数比较。

其中，`p4_walk` 内部又分为中点计算、左右递归以及返回值处理三个部分。

本关与普通二分查找的区别是：程序不仅要找到目标值，还会在递归返回时进行额外计算，最终将计算结果作为正确答案。

4.3 各部分逐步突破

4.3.1 输入检查与参数处理

首先观察：

```
mov $0x1,%esi
call valid_u32_fields
```

Fence 32

说明本关要求输入一个无符号整数。

随后程序对 `challenge` 进行处理：

```
mov %r12,%rcx
mov %r12,%rax
mov %r12d,%edi
xor %esi,%esi
shr $0x20,%rax
shr $0x7,%rcx
and $0x7f,%edi
mov $0x7f,%edx
xor %eax,%ecx
call p4_walk
```

Fence 33

根据寄存器传参规则，可以得到 `p4_walk` 的四个参数：

```
target = (uint32_t)challenge & 0x7f;
low = 0;
high = 127;
key = (uint32_t)(challenge >> 7)
    ^ (uint32_t)(challenge >> 32);
```

Fence 34

因此，程序在 `[0,127]` 区间中搜索由 `challenge` 低 7 位确定的目标值。

4.3.2 中点计算与二分查找识别

在 `p4_walk` 开头发现：

```
mov %edx,%edi
sub %esi,%edi
shr $1,%edi
add %esi,%edi
```

Fence 35

其中 `%esi` 和 `%edx` 分别对应区间下界和上界。

逐条还原后得到：

```
mid = low + ((high - low) >> 1);
```

Fence 36

这正是二分查找的中点计算公式：

$$mid = low + \left\lfloor \frac{high - low}{2} \right\rfloor$$

由此可以确定函数采用二分查找算法。

4.3.3 递归分支分析

接下来观察到：

```
cmp %eax,%edi
je 402320

...

cmp %edi,%eax
jb 402300
```

Fence 37

此时 `%eax` 保存目标值，`%edi` 保存中点。

第一组比较判断目标值是否等于中点，若相等，则直接跳转至递归终止部分。

第二组比较使用 `jb` 判断目标值是否小于中点。结合后续对区间边界的修改，可以确定：

`target < mid`：递归搜索左半区间 `[low, mid-1]`。

`target > mid`：递归搜索右半区间 `[mid+1, high]`。

`target == mid`：找到目标，结束递归。

这与前面判断的二分查找结构一致。

4.3.4 递归返回值的计算

与普通二分查找不同，`p4_walk` 在找到目标以及递归返回时都会进行额外运算。

当目标等于中点时：

```
lea 0xae(%rdi),%eax
xor %ecx,%eax
ret
```

Fence 38

对应：

```
return (mid + 0xaeu) ^ key;
```

Fence 39

当目标大于中点时，程序向右递归，并在返回后执行：

```
lea (%rax,%rax,2),%eax
xor %ebx,%eax
xor $0x10,%eax
```

Fence 40

其中 `%ebx` 保存 `mid + key`，因此对应：

```
return ((3u * result) ^ (mid + key)) ^ 0x10u;
```

Fence 41

而目标小于中点时，返回值的计算方式基本相同，只是最后异或的常数变为 `0x12`：

```
return ((3u * result) ^ (mid + key)) ^ 0x12u;
```

Fence 42

因此，递归返回值不仅取决于目标值，还与搜索过程中经过的中点及分支方向有关。

至此，`p4_walk` 的完整递归逻辑已经还原。

4.4 最终代码实现

根据上述分析，使用递归函数模拟原程序的二分查找及返回值计算。

```
static uint32_t p4_walk_helper(uint32_t target, uint32_t low, uint32_t
high, uint32_t key)
{
    uint32_t mid = low + ((high - low) >> 1);
    if (target == mid)
        return (mid + 0xaeu) ^ key;
    uint32_t temp = mid + key;
    if (target < mid) {
        uint32_t result = p4_walk_helper(target, low, mid - 1, key);
        return ((3u * result) ^ temp) ^ 0x12u;
    }
    else {
        uint32_t result = p4_walk_helper(target, mid + 1, high, key);
        return ((3u * result) ^ temp) ^ 0x10u;
    }
}
int solve_phase_4(uint64_t challenge, char *out, size_t cap)
{
    uint32_t values[MAX_FIELDS] = {0};
```

```
    size_t count = 1;
    uint32_t target = (uint32_t)challenge & 0x7fu;
    uint32_t key = (uint32_t)(challenge >> 7)
                 ^ (uint32_t)(challenge >> 32);
    values[0] = p4_walk_helper(target, 0, 127, key);
    return write_answer(out, cap, values, count);
}
```

Fence 43

辅助函数 `p4_walk_helper` 用于还原原程序的递归计算，而 `solve_phase_4` 根据 `challenge` 构造参数，调用辅助函数并输出最终答案。

4.5 运行结果截图

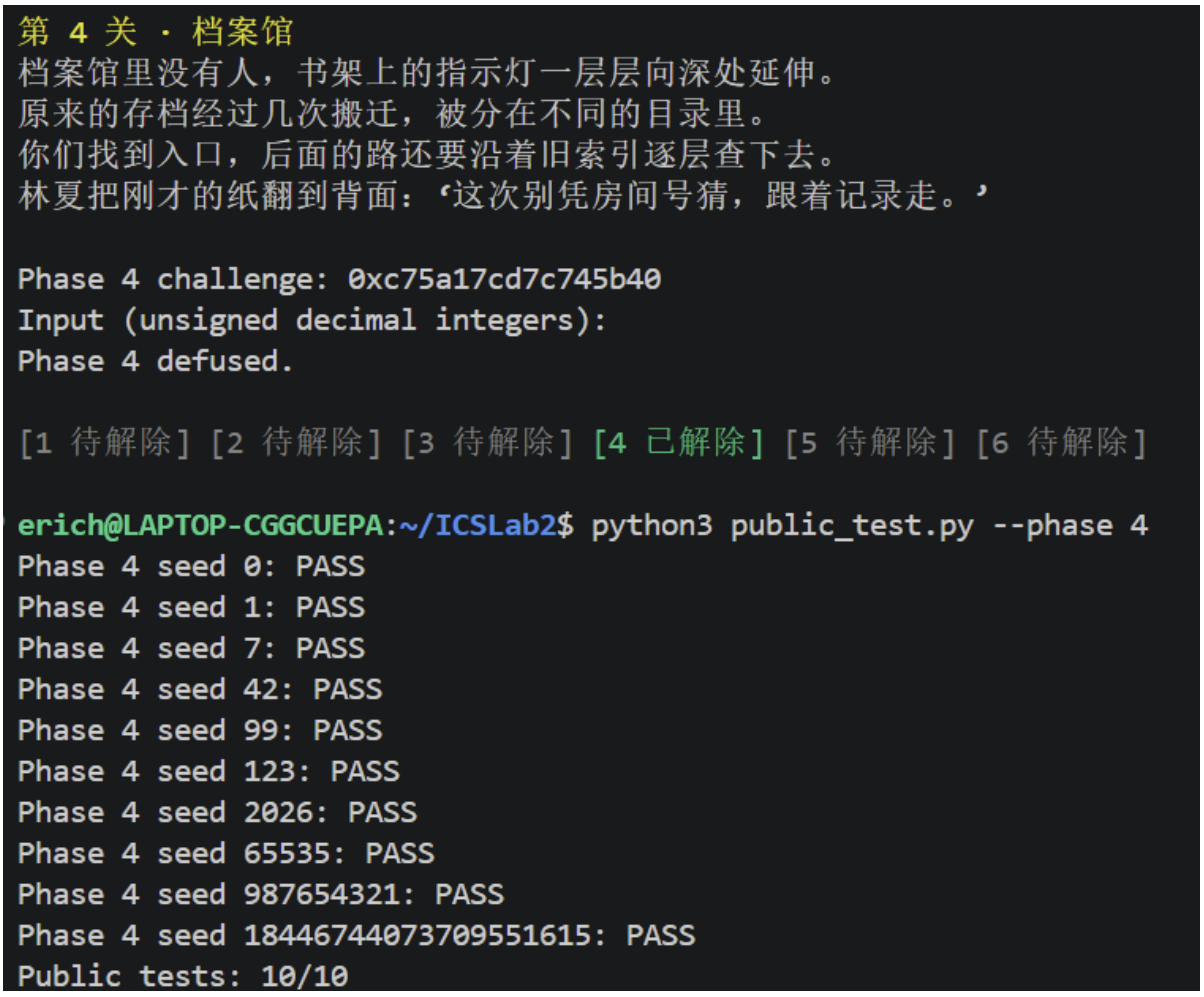


Figure 4

5. Q5

5.1 破题思路

我先阅读实验手册，了解到第五关与链表有关，因此在分析汇编时，主要关注指针操作、内存寻址以及循环结构。

使用 GDB 的 `disas phase_05` 命令查看汇编后，发现本关需要输入两个整数，而主要的计算过程集中在 `p5_traverse` 函数中，于是进一步对其反汇编。

进入 `p5_traverse` 后，注意到大量带括号的内存寻址指令，例如 `0x8(%rdx)`、`0xc(%rdx)` 和 `(%rdx)`。结合链表的特点，推测这些操作分别对应节点成员访问和 `next` 指针的更新。

随后通过 GDB 查看节点所在的内存区域，确定结构体布局、链表节点之间的连接关系及各节点的数据。最后根据循环中的运算还原遍历过程，并编写求解器。

5.2 结构剖析

根据 `phase_05` 和 `p5_traverse` 的汇编，可以将第五关划分为四个部分：

1. 解析两个无符号 32 位整数。
2. 根据 `challenge` 确定链表起点和遍历次数。
3. 按照 `next` 指针依次访问节点，并进行累计运算。
4. 分别比较函数返回值和通过指针写回的结果。

其中，`p5_traverse` 同时使用寄存器返回值和指针参数输出两个结果，这是与前几关不同的地方。

5.3 各部分逐步突破

5.3.1 输入检查与关键函数

依旧公式化观察：

```
mov $0x2,%esi
call valid_u32_fields
```

Fence 44

结合后续 `sscanf` 返回值与 2 的比较，可知本关需要输入两个整数。

随后发现：

```
lea -0x14(%rbp),%rsi
mov %r12,%rdi
call p5_traverse
```

Fence 45

这里 `%rdi` 传入 `challenge`，而 `%rsi` 保存一个局部变量的地址。

结合函数调用结束后的两次比较，可以判断 `p5_traverse` 会通过 `%eax` 返回第一个计算结果，并通过指针写回第二个结果。

5.3.2 结构体布局与链表识别

观察 `p5_traverse` 中的内存访问：

```
mov 0x405b20(,%rax,8),%rdx
mov 0x8(%rdx),%esi
xor 0xc(%rdx),%eax
mov (%rdx),%rdx
```

Fence 46

第一条指令通过下标访问一张指针表，获取链表的起始节点地址。

后面三条指令分别读取节点中偏移 8 字节、12 字节和 0 字节处的数据。

结合 64 位指针占 8 字节、`uint32_t` 占 4 字节的特点，可以还原节点结构：

```
struct Node {
    struct Node *next;
    uint32_t value;
    uint32_t key;
};
```

Fence 47

其中：

`next` 位于偏移 0，保存下一节点地址。

`value` 位于偏移 8，保存节点数值。

`key` 位于偏移 12，参与累计计算。

有一个很明显的代码：

```
mov (%rdx),%rdx
```

Fence 48

等价于：

```
p = p->next;
```

Fence 49

说明程序会沿链表依次访问后续节点，与实验手册中的提示一致。

5.3.3 链表数据恢复

为了还原实际的链表结构，使用 GDB 查看内存：

```
x/8gx 0x405b20
x/64gx 0x405b60
```

Fence 50

第一条命令用于查看八个链表头指针，第二条命令用于读取节点的实际数据。

例如，位于 `0x405b60` 的节点中：

```
next  = 0x405c00
value = 0x071c
key   = 0xc3f5
```

Fence 51

由此可以将原来的内存地址转换成数组下标，在求解程序中重新构建链表。

5.3.4 遍历过程与计算关系

进一步分析 `p5_traverse`，发现程序根据 `challenge` 的不同位段确定起始节点和移动次数：

```
index = challenge & 7u;
steps = (challenge >> 3) & 3u;
```

Fence 52

因此，`index` 取值为 0～7，决定使用哪个链表头；`steps` 取值为 0～3，决定向后移动多少次。

程序首先利用起始节点的数据初始化累计结果：

```
result = (uint32_t)(challenge >> 6)
       ^ (uint32_t)(challenge >> 32);

result ^= node->key;
result ^= 0x2865u;
result += node->value;
```

Fence 53

之后根据 `steps` 沿链表遍历：

```
for (int i = 0; i < steps; i++) {
    node = node->next;
    result ^= node->key;
    result += node->value;
}
```

Fence 54

每次遍历都会先移动到下一节点，再使用该节点的 `key` 和 `value` 更新结果。

最后返回累计值，同时输出最后访问节点的 `value`。

因此本关的两个答案分别为：

```
values[0] = result;
values[1] = node->value;
```

Fence 55

逻辑还原完毕。

5.4 最终代码实现

由于 `defuser.c` 不能直接访问炸弹内部的节点地址，因此使用数组下标代替原来的指针，并根据 GDB 获取的数据重建链表。

```
int solve_phase_5(uint64_t challenge, char *out, size_t cap)
{
    typedef struct {
        int next;
        uint32_t value;
        uint32_t key;
    } P5Node;
    static const P5Node p5_nodes[12] = {
        { 10,     0x071cu,    0xc3f5u },
        {  4,     0x021au,    0x1073u },
        {  7,     0x0a47u,    0xdb1fu },
        {  0,     0x0349u,    0xe266u },
        {  5,     0x0548u,    0x6689u },
        {  9,     0x005cu,    0xa66bu },
        { -1,     0x0871u,    0xaccau },
        {  3,     0x0b67u,    0x37aau },
        {  2,     0x0654u,    0x99d9u },
        {  8,     0x017bu,    0xa3a0u },
        {  6,     0x0935u,    0x5dbdu },
```

```
        {  1,     0x0476u,    0xfafeu }
    };
    static const int heads[8] = {
        11, 1, 4, 5, 9, 8, 2, 7
    };
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;
    uint32_t index = (uint32_t)challenge & 7u;
    uint32_t steps = (uint32_t)(challenge >> 3) & 3u;
    int p = heads[index];
    uint32_t result = (uint32_t)(challenge >> 6)
                    ^ (uint32_t)(challenge >> 32);
    result ^= p5_nodes[p].key;
    result ^= 0x2865u;
    result += p5_nodes[p].value;
    for (uint32_t i = 0; i < steps; i++) {
        p = p5_nodes[p].next;
        result ^= p5_nodes[p].key;
        result += p5_nodes[p].value;
    }
    values[0] = result;
    values[1] = p5_nodes[p].value;
    return write_answer(out, cap, values, count);
}
```

Fence 56

通过这种方式，将原程序基于真实内存地址的链表操作转换成数组下标访问，保留了相同的节点连接关系和计算逻辑。

5.5 运行结果截图



Figure 5

6. Q6

6.1 破题思路

由于已经完成前五关，因此输入检查和参数传递部分比较容易理解，很快就确定本关要求输入一个整数，核心计算集中在 `p6_dispatch` 中。

继续反汇编 `p6_dispatch`，发现其中存在两次间接 `call`，并且都通过地址 `0x405dc0` 附近的内存读取函数地址。这与第三关遇到的跳转表比较相似，因此推测程序使用了函数指针数组。

随后按照汇编执行顺序，分析 `challenge` 的位运算、函数索引的提取及两次函数调用之间的数据传递。最后分别还原六个计算函数，将其组合成完整的求解算法。

本题比较简单，顺着做就行了。

6.2 结构剖析

整个 `phase_06` 可以分为三个部分：

1. 使用 `valid_u32_fields` 和 `sscanf` 解析一个无符号 32 位整数。
2. 从 `challenge` 中提取计算参数和两个函数索引。
3. 通过函数指针表依次调用两个计算函数，并将最终返回值与输入整数比较。

其中，`p6_dispatch` 是本关的核心，它并不直接完成所有运算，而是根据 `challenge` 选择两个函数，并将它们的计算结果组合起来。

6.3 各部分逐步突破

6.3.1 输入检查与关键函数

```
mov $0x1,%esi
call valid_u32_fields
```

Fence 57

说明本关需要输入一个整数。

随后程序调用：

```
mov %r12,%rdi
call p6_dispatch
cmp %eax,-0x14(%rbp)
sete %al
```

Fence 58

这里 `%r12` 保存 `challenge`，`p6_dispatch` 的返回值与输入整数进行比较。因此只需要还原 `p6_dispatch(challenge)` 的计算逻辑。

6.3.2 计算参数与函数索引

在 `p6_dispatch` 中，首先发现：

```
mov %rdi,%rbx
shr $0x6,%rdi
mov %rbx,%rax
shr $0x20,%rax
xor %eax,%edi
```

Fence 59

这部分将 `challenge` 分别右移 6 位和 32 位，再对低 32 位进行异或，得到计算参数：

```
uint32_t x = (uint32_t)(challenge >> 6)
           ^ (uint32_t)(challenge >> 32);
```

Fence 60

随后：

```
mov %rbx,%rax
and $0x38,%ebx
and $0x7,%eax
```

Fence 61

`0x7` 的二进制为 `111`，因此很容易判断第一项是在提取 `challenge` 最低 3 位。

对于 `0x38`，将其转换成二进制后得到：

```
0x38 = 00111000
```

Fence 62

可以发现它保留的是从右往左数第 4 ~ 6 位。这个操作本质上等价于先将 `challenge` 右移 3 位，再保留最低 3 位，只不过原汇编中保留的是乘以 8 后的地址偏移量。

最终得到两个函数索引：

```
uint32_t k0 = (uint32_t)challenge & 7u;
uint32_t k1 = (uint32_t)(challenge >> 3) & 7u;
```

Fence 63

6.3.3 函数指针表分析

继续观察两次间接调用：

```
call *0x405dc0(,%rax,8)
xor $0x9864,%eax
mov %eax,%edi
call *0x405dc0(%rbx)
```

Fence 64

第一次调用使用 `0x405dc0 + 8*k0` 定位函数指针。

第二次调用使用 `0x405dc0 + rbx` 定位函数指针。由于 `%rbx` 中保存的是 `challenge & 0x38`，即 `8*k1`，所以两次调用实际上都在访问同一个函数指针数组。

通过 GDB 查看：

```
x/8ga 0x405dc0
```

Fence 65

得到函数索引与实际函数的对应关系：

索引	对应函数
0	p6_op_3
1	p6_op_5
2	p6_op_0
3	p6_op_2
4	p6_op_1
5	p6_op_2
6	p6_op_1
7	p6_op_4

Table 3

这说明本关采用了函数指针数组，通过不同索引选择计算方式。

6.3.4 各计算函数的还原

分别反汇编 `p6_op_0` 至 `p6_op_5`，发现这些函数都比较短，比较简单，主要由移位、异或、加法和 `lea` 组成。

因此可以直接还原为以下 C 语言表达式，设函数参数为 `x`：

函数	计算结果
p6_op_0	`(x >> 2) | (x << 30)`
p6_op_1	`((x >> 7) ^ x) + 0xf8fc`
p6_op_2	`((x << 7) + 0x1aa7) ^ (3*x)`
p6_op_3	`(x ^ 0xc09c) + (x << 5)`
p6_op_4	`((x << 1) | (x >> 31)) ^ 0x3362`
p6_op_5	`(x + 0x9cf4) ^ (x >> 5)`

Table 4

6.3.5 组合计算关系

根据 `p6_dispatch` 中的两次间接调用，可以还原完整过程：

```
uint32_t y = p6_ops[k0](x);
y ^= 0x9864u;
uint32_t z = p6_ops[k1](y);
return z + 0x9ff5u;
```

Fence 66

6.4 最终代码实现

```
static uint32_t p6_apply(int op, uint32_t x)
{
    switch (op) {
        case 0:
            return (x >> 2) | (x << 30);
        case 1:
            return ((x >> 7) ^ x) + 0xf8fcu;
        case 2:
            return ((x << 7) + 0x1aa7u) ^ (3u * x);
        case 3:
            return (x ^ 0xc09cu) + (x << 5);
        case 4:
            return ((x << 1) | (x >> 31)) ^ 0x3362u;
        case 5:
            return (x + 0x9cf4u) ^ (x >> 5);
        default:
            return 0;
    }
}
int solve_phase_6(uint64_t challenge, char *out, size_t cap)
{
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;
    static const int ops[8] = {
        3, 5, 0, 2, 1, 2, 1, 4
    };
    uint32_t x = (uint32_t)(challenge >> 6)
```

```
                ^ (uint32_t)(challenge >> 32);
    uint32_t k0 = (uint32_t)challenge & 7u;
    uint32_t k1 = (uint32_t)(challenge >> 3) & 7u;
    uint32_t y = p6_apply(ops[k0], x);
    y ^= 0x9864u;
    uint32_t z = p6_apply(ops[k1], y);
    values[0] = z + 0x9ff5u;
    return write_answer(out, cap, values, count);
}
```

Fence 67

这里使用 `p6_apply` 模拟原来的六个计算函数，再通过 `ops` 数组还原函数指针表的排列关系。

6.5 运行结果截图

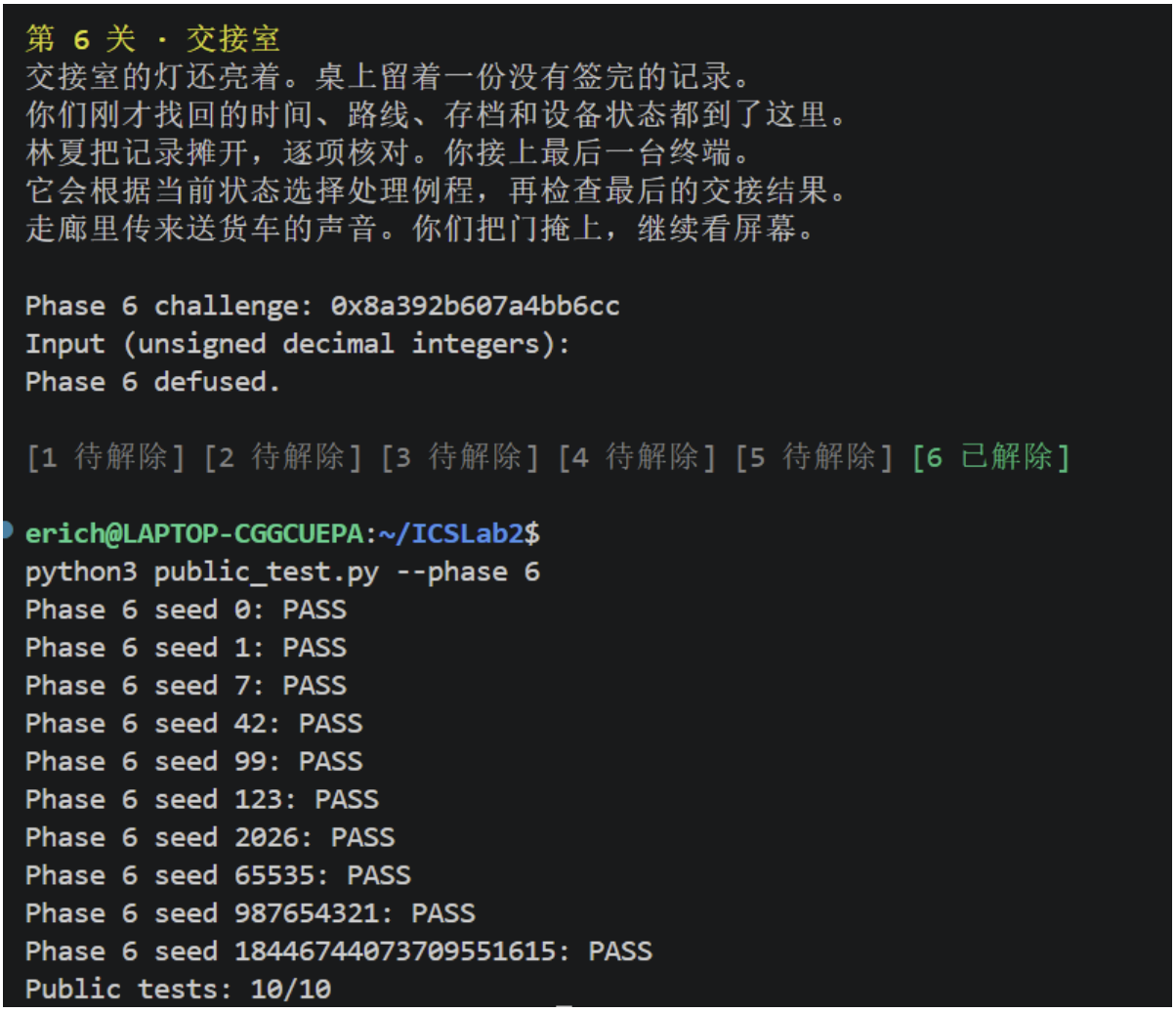


Figure 6

7. Honor

7.1 破题思路

根据实验手册的提示，使用 GDB 分别查看 `secret_gate`、`secret_copy` 和 `secret_phase` 的汇编代码。

在分析 `secret_gate` 时，发现程序进行了大量栈空间操作，而且出现了一条比较奇怪的指令：

```
cmpl $0x4e45504f,-0xb0(%rbp)
```

起初不太明白为什么要将栈上的数据与这样一个十六进制常数比较。后来查阅 ASCII 编码，并考虑到 x86-64 使用小端序存储数据，才发现 `0x4e45504f` 在内存中对应的字符竟然是 `OPEN`。

这一发现很有意思，也让我意识到本关与之前六关完全不同，不是简单的整数验证，而是需要构造具有特定内存布局的字符串。

于是将分析重点转向栈上的地址关系。通过计算不同地址之间的偏移，确定 `OPEN` 在输入字符串中的位置，再结合 `secret_copy` 的复制逻辑和 `secret_phase` 的验证条件，逐步还原完整的口令格式。

7.2 结构剖析

根据反汇编结果，Honor 可以分为三个主要部分：

1. `secret_gate`：初始化栈空间、检查输入长度及特殊标记。
2. `secret_copy`：将输入字符串逐字节复制到栈上的缓冲区。
3. `secret_phase`：读取特殊标记之后的三个整数，并根据 `challenge` 和 `context` 验证。

其中，`secret_gate` 主要负责构造和检查内存布局，`secret_phase` 则负责最终的数值验证。

7.3 各部分逐步突破

7.3.1 栈空间初始化与内存布局

观察 `secret_gate` 中的指令：

```
lea -0xd0(%rbp),%r12
mov %r12,%rdi
mov $0x14,%ecx
xor %eax,%eax
rep stos %rax,(%rdi)
movl $0x0,(%rdi)
```

Fence 69

其中 `%r12` 保存栈上缓冲区的起始地址 `rbp-0xd0`。

`rep stos` 将连续的 20 个 8 字节区域清零，之后再清零 4 字节，总计初始化 164 字节。

随后发现两个关键地址：

```
cmpl $0x4e45504f,-0xb0(%rbp)
lea -0xac(%rbp),%rdi
call secret_phase
```

Fence 70

为了弄清这些地址的关系，我打了草稿，计算并记录它们相对于缓冲区起点的偏移：

```
缓冲区起点：rbp - 0xd0

OPEN 检查位置：
0xd0 - 0xb0 = 0x20 = 32 字节

secret_phase 参数位置：
0xd0 - 0xac = 0x24 = 36 字节
```

Fence 71

据此绘制出以下栈内存布局：

```
                    高地址
                  rbp - 0x2c
             ┌──────────────────────┐
             │                      │
             │ 后续输入内容         |
             │                      |
             | 从偏移 36 开始       |
             ├──────────────────────┤
rbp-0xac     | 三个整数的文本       | ← secret_phase
             ├──────────────────────┤
rbp-0xb0     | O P E N              | ← 特殊标记
             | 共 4 字节            |
             ├──────────────────────┤
             |                      |
             | 前 32 字节           |
             | 填充区域             |
             |                      |
rbp-0xd0     └──────────────────────┘
                    低地址
```

Fence 72

由此可以确定，程序将字符串复制到 `rbp-0xd0` 开始的位置，在偏移 32 处检查四字节标记，并将偏移 36 开始的内容交给 `secret_phase`。

7.3.2 特殊标记 OPEN 的发现

重点分析：

```
cmpl $0x4e45504f,-0xb0(%rbp)
jne 4026fb
```

Fence 73

这里的 `cmpl` 实际上是在比较一个 32 位整数，并不是直接调用字符串比较函数。

由于 x86-64 采用小端序，常数 `0x4e45504f` 对应的内存字节顺序是：

```
十六进制：  4f  50  45  4e
ASCII：    O   P   E   N
```

Fence 74

因此，只要将 `OPEN` 放在输入字符串的第 33～36 个字符位置，就可以满足这一比较条件。

这是本关比较巧妙的设计，程序表面上在比较一个整数，实际上却通过内存中的字符编码检查了一个字符串标记，我一开始在想这个十六进制好像和之前位运算有关的十六进制长得也不一样，那这个十六进制数究竟在做什么，后面想到了可能是ASCII码对应字符串，于是去查表，惊喜发现是open。

7.3.3 字符串复制与长度检查

反汇编 `secret_copy`：

```
movzbl (%rsi,%rax,1),%edx
mov %dl,(%rdi,%rax,1)
add $0x1,%rax
test %dl,%dl
jne 4026a0
```

Fence 75

可以发现，这段代码通过下标逐字节读取输入，再写入目标缓冲区，直到遇到字符串结束符 `\0`。

因此，其功能相当于：

```
void secret_copy(char *dest, const char *src)
{
    size_t i = 0;
    do {
        dest[i] = src[i];
    } while (src[i++] != '\0');
}
```

Fence 76

同时，`secret_gate` 还使用 `strlen` 检查长度：

```
call strlen@plt
sub $0x24,%rax
cmp $0x7f,%rax
jbe 402710
```

Fence 77

其中 `0x24 = 36`，`0x7f = 127`。通过无符号比较，可以确定允许的输入长度范围为 36 ~ 163 字节。

结合前面的栈图，最终确定 Honor 输入具有以下结构：

```
[32字节填充]OPEN[三个无符号整数]
```

Fence 78

其中前 32 字节可以使用普通 ASCII 字符填充，后面的三个整数需要满足 `secret_phase` 的验证条件。

7.3.4 secret_phase 的数值计算

完成字符串布局分析后，继续查看 `secret_phase`。

首先：

```
mov $0x3,%esi
call valid_u32_fields
```

Fence 79

可以确定特殊标记后面需要输入三个无符号 32 位整数，记为 `a`、`b`、`c`。

第一个整数通过以下指令确定：

```
mov %r13d,%ecx
and $0x3,%ecx
cmp %ecx,-0x2c(%rbp)
```

Fence 80

由于 `%r13` 保存 `context`，因此：

```
a = (uint32_t)context & 3u;
```

Fence 81

后面两项涉及 `challenge`、`context` 的移位及异或运算，其中还调用了 `secret_mix`。

该函数的汇编为：

```
mov %edi,%eax
ror $0x3,%eax
xor $0x5bf2,%eax
ret
```

Fence 82

其中 `ror` 表示 32 位循环右移，由于这是32位整数，所以我们可以用位运算式来表示：

```
mix = ((x >> 3) | (x << 29)) ^ 0x5bf2u;
```

Fence 83

将其他寄存器运算整理后，可以得到：

```
uint32_t x = (uint32_t)(challenge >> 32)
           ^ (uint32_t)challenge;

uint32_t y = (uint32_t)(context >> 32)
           ^ (uint32_t)context;

uint32_t k = (a * 0x9a55u)
           ^ (uint32_t)(context >> 16);

uint32_t t = x ^ y;
uint32_t mix = ((t >> 3) | (t << 29)) ^ 0x5bf2u;

uint32_t b = mix ^ k;
uint32_t c = b ^ (y + a + 0xc1a7u + (x >> 3));
```

Fence 84

至此，完整的字符串结构和三个整数的计算公式均已确定。

7.4 最终代码实现

根据前面的栈布局和数值计算，在 `solve_secret` 中使用 `snprintf` 直接生成第七行字符串。

```
int solve_secret(uint64_t challenge, uint64_t context,
                 char *out, size_t cap)
{
    uint32_t a = (uint32_t)context & 3u;
    uint32_t x = (uint32_t)(challenge >> 32)
               ^ (uint32_t)challenge;
    uint32_t y = (uint32_t)(context >> 32)
               ^ (uint32_t)context;
    uint32_t k = (a * 0x9a55u)
               ^ (uint32_t)(context >> 16);
```

```
    uint32_t t = x ^ y;
    uint32_t mix = ((t >> 3) | (t << 29)) ^ 0x5bf2u;
    uint32_t b = mix ^ k;
    uint32_t c = b ^ (y + a + 0xc1a7u + (x >> 3));
    int n = snprintf(out, cap,
        "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN%u %u %u",
        a, b, c);
    if (n < 0 || (size_t)n >= cap)
        return 1;
    return 0;
}
```

Fence 85

7.5 运行结果截图

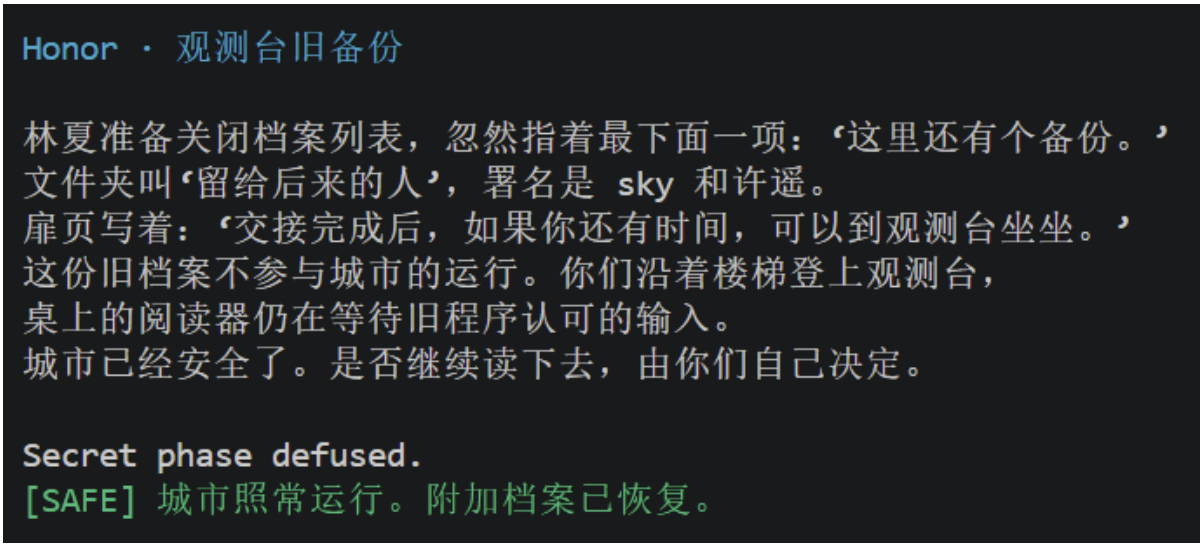


Figure 7

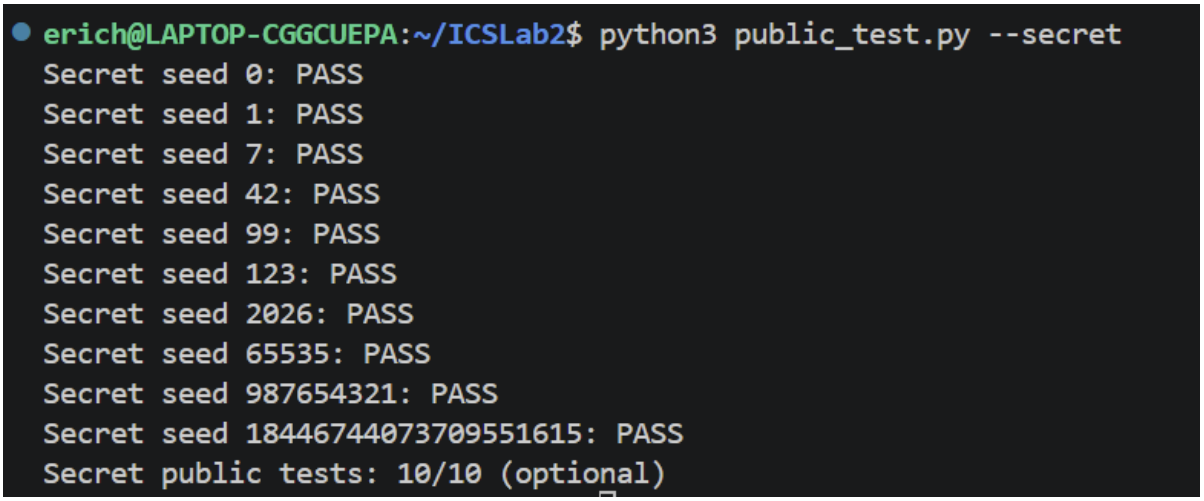


Figure 8

8. 后言

这次作业我个人写下来觉得前期是很痛苦的，因为对汇编代码实在不熟悉，课上的演示代码并不复杂，容易看懂，但是实操翻译真正的程序发现有时候无从下手。

我觉得很大的问题在我不知道那些寄存器对应的参数在c语言中究竟代表了什么，所以我会去问ai，他会告诉我哪个是input，哪个是challenge，哪个是栈指针等等。

前期我会使用ai辅助我，将代码分成一块一块的形式，我逐个突破。做了几道题后发现大部分汇编结构相近，语句重复度很高，同时对操作也熟悉了，慢慢能自己翻译一大半，但是还是无法保证全部正确，于是询问ai让他帮我指出错误，我再试着修改，得到了最终答案。

每道题做完我会自己先整理思路，写在word里发给ai让ai帮我润色实验报告，但是整体报告的结构都是我先自己写好的，它主要是帮我将缺乏逻辑和过于口语化的语言翻译成书面且拥有逻辑的语言，并且补充对应的汇编代码与c语言代码。我也会让它帮忙生成流程图，使我的报告更有可读性。

我认为做完这个lab，我对于汇编语言的理解提升了不少，虽然痛苦但能学到知识。